CAHIER DES CHARGES COMPLET — PROJET FIL ROUGE

Par Robert DIASSÉ

Système de Réservation de Services Médicaux

1. Présentation générale du projet

Le projet consiste à réaliser une **application web moderne** permettant :

- aux patients de consulter des services médicaux,
- aux patients de réserver un créneau,
- aux administrateurs de gérer les services et les réservations,
- aux médecins (ou personnel médical) d'avoir un accès restreint à certaines informations,
- le tout avec une authentification et une gestion des rôles.

L'objectif est de créer une application simple mais réaliste, respectant les bonnes pratiques d'un backend professionnel en Laravel.

---

2. Objectifs

Ce projet doit permettre de maîtriser :

- la création d'une API et d'un backend avec Laravel,
- l'architecture MVC et l'utilisation des routes, controllers, modèles, vues,
- l'intégration de Blade et des layouts,
- l'utilisation d'un template d'administration,
- les migrations, seeders, factories,
- la manipulation d'Eloquent ORM,
- les relations entre tables,
- l'authentification (Laravel Breeze ou Jetstream),
- la gestion des rôles,
- un CRUD complet,
- le fonctionnement des réservations,
- la validation des formulaires,
- l'organisation d'un projet complet en Laravel.

---

3. Définition du domaine : services médicaux

Un **service médical** est une prestation que peut offrir une clinique, un cabinet ou un personnel de santé.

Exemples :

- consultation générale
- consultation pédiatrique
- suivi diabétique
- soins infirmiers
- imagerie médicale
- vaccination
- ECG / ECG de repos
- test auditif

Chaque service peut :

- être réservé par un patient,
- être géré par un administrateur,
- avoir un prix,
- avoir une description,
- avoir un temps moyen,
- être rattaché à un médecin (optionnel et simplifié).

---

4. Utilisateurs et rôles

L'application comporte **trois rôles** simples :

1) **Administrateur**

- peut gérer les services médicaux (CRUD complet),
- peut gérer les utilisateurs,
- peut voir toutes les réservations,
- peut annuler une réservation,
- peut attribuer un médecin à un service.

2) **Médecin / Prestataire**

- peut voir ses propres réservations,
- peut mettre à jour le statut d'une réservation (ex : confirmée / effectuée),
- ne peut pas ajouter/modifier des services,
- ne peut pas voir les réservations des autres médecins.

3) **Patient**

- peut consulter la liste des services,
- peut créer une réservation,

- peut voir ses propres réservations,
- peut annuler une réservation (avant un certain statut).

---

5. Fonctionnalités (liste exhaustive, simple et réalisable)

5.1 Authentification

- Inscription
- Connexion
- Déconnexion
- Mot de passe oublié (optionnel)
- Rôles (admin / medecin / patient)

Laravel Breeze suffira.

5.2 Gestion des services médicaux (Admin)

L'administrateur peut :

- ajouter un service
- modifier un service
- supprimer un service
- voir la liste des services
- désactiver/activer un service
- attribuer un service à un médecin (optionnel)

Champs d'un service :

- id
- titre
- description
- prix
- durée (ex : 30 minutes)
- statut (actif/inactif)
- id_medecin (nullable)

5.3 Gestion des réservations

Modèle mental simple :

Un patient choisit un service → choisit un créneau horaire → confirme.

Actions possibles :

- le patient peut réserver un service

- le médecin peut consulter ses réservations

- le médecin peut mettre un statut :
  - en attente
  - confirmée
  - annulée
  - effectuée
- l'admin peut voir toutes les réservations
- l'admin peut annuler une réservation

Champs d'une réservation :

- id
- id_patient
- id_service
- date_reservation
- heure_reservation
- statut (« en attente », « confirmée », « annulée », « effectuée »)
- commentaire optionnel

(Le système de disponibilité avancée n'est pas inclus pour garder le projet simple.)

5.4 Gestion des utilisateurs (Admin)

- voir la liste des utilisateurs
- attribuer un rôle
- désactiver un utilisateur
- voir le nombre de réservations par utilisateur (optionnel)

---

6. Schéma de la base de données

Voici une version simple et complète adaptée au projet :

```
users
  id
  name
  email
  password
  role (admin / medecin / patient)
  created_at
  updated_at

services
  id
  titre
  description
  prix
```

```
    duree
    statut (actif/inactif)
    medecin_id (nullable)
    created_at
    updated_at

  reservations
    id
    user_id (patient)
    service_id
    date_reservation
    heure_reservation
    statut
    created_at
    updated_at
```

---

7. Relations Eloquent

Laravel ORM (Eloquent) représentera les relations suivantes :

User ↔ Reservation

- Un utilisateur (patient) a plusieurs réservations
- Une réservation appartient à un utilisateur

```php
// User.php
public function reservations() {
    return $this->hasMany(Reservation::class);
}

// Reservation.php
public function user() {
    return $this->belongsTo(User::class);
}
```

Service ↔ Reservation

- Un service peut avoir plusieurs réservations
- Une réservation appartient à un service

Medecin ↔ Service

- Un médecin peut avoir plusieurs services
- Un service peut être attribué à un médecin (optionnel)

---

8. Use cases (scénarios d'utilisation)

Patient

1. naviguer sur le site
2. voir la liste des services
3. consulter les détails d'un service
4. réserver un créneau simple (date + heure)
5. voir ses réservations
6. annuler une réservation en attente

Médecin

1. se connecter
2. voir uniquement ses services
3. voir ses réservations
4. changer le statut d'une réservation

Administrateur

1. gérer les services (CRUD)
2. affecter un médecin
3. voir toutes les réservations
4. mettre à jour le statut d'une réservation
5. gérer les utilisateurs
6. gérer les rôles

---

9. Périmètre prévu (important)

Le projet restera simple et réalisable rapidement :

- pas de paiement en ligne,
- pas de disponibilité complexe,
- pas de gestion de calendrier avancé,
- pas de rôle super-admin,
- pas de permissions ACL avancées,
- design simple basé sur un template admin (SB Admin 2 / AdminLTE).

---

10. Technologie et outils utilisés

- **Laravel 12**
- **PHP 8.2+**
- **Blade (vues)**
- **MySQL / MariaDB / SQLite**
- **Tailwind + Breeze (auth)**
- **AdminLTE ou SB Admin pour l'admin**

- **Eloquent ORM**
- **Artisan**

---

11. Architecture Laravel recommandée pour ce projet

```
app/
 ├── Http/
 │    ├── Controllers/
 │    │      ├── ServiceController.php
 │    │      ├── ReservationController.php
 │    │      └── AdminController.php
 │    └── Middleware/
 ├── Models/
 │    ├── User.php
 │    ├── Service.php
 │    └── Reservation.php
resources/
 ├── views/
 │    ├── layouts/
 │    ├── services/
 │    ├── reservations/
 │    ├── admin/
routes/
 ├── web.php
 ├── api.php
database/
 ├── migrations/
 ├── seeders/
```

---

12. Validation fonctionnelle (ce qui doit absolument fonctionner)

Fonctionnalités front simples :

- affichage des services
- formulaire de réservation
- tableau des réservations

Côté admin :

- CRUD complet des services
- gestion des réservations
- gestion des utilisateurs
- assignation des rôles

Authentification et rôles :

- connexion / inscription
- middleware de rôle
- redirection automatique selon le rôle

---

13. Étapes du développement

1. Installation du projet Laravel
2. Mise en place du template frontend
3. Mise en place du template admin
4. Migrations (users, services, reservations)
5. Seeders pour exemples
6. Authentification (Laravel Breeze)
7. Gestion des rôles via middleware
8. CRUD Services (admin)
9. Réservation (patient)
10. Dashboard médecin
11. Dashboard admin
12. Tests rapides
13. Déploiement local final

---

14. Interface utilisateur (attendue)

Tableau de bord admin :

- nombre de services
- nombre de réservations
- graphique simple (optionnel)

Liste des services :

- tableau
- boutons (modifier, supprimer)

Formulaire de service :

- titre, description, prix, durée, médecin, statut

Réservation :

- choix du service
- sélection date + heure
- confirmation

Mes réservations patient :

- tableau filtré

Réservations médecin :

- tableau
- bouton changer statut

---

15. Contraintes et simplicité du projet

Pour que ce projet reste **réalisable rapidement**, les choix suivants sont faits :

- système de disponibilité simple (date + heure libre)
- aucune gestion d'agenda complexe
- pas de notifications par mail
- interface admin déjà existante via template
- routes web (HTML) seulement, APIs optionnelles
- authentification fournie par Breeze

Ces contraintes permettent d'apprendre **Laravel à 100 % sans se noyer dans la complexité** d'un vrai système hospitalier.

Voici **la liste complète, officielle et définitive des routes attendues** pour le projet fil rouge *« Système de Réservation de Services Médicaux »*.

Ces routes sont **cohérentes**, **limitées au périmètre du cahier des charges**, **simples**, **logiques**, et entièrement compatibles avec une architecture Laravel .

Je sépare les routes par rôle :

- Routes publiques
- Routes authentifiées
- Routes patient
- Routes médecin
- Routes administrateur

Puis je fournis le **fichier web.php final attendu**.

---

1. Routes publiques (sans authentification)

```
GET   /                # page d'accueil simple
GET   /services        # liste des services visibles publiquement
GET   /services/{id}   # détails d'un service
```

Ces pages sont accessibles sans être connecté — uniquement consultation.

---

2. Routes d'authentification (Laravel Breeze)

Ces routes sont générées automatiquement :

```
/register
/login
/logout
/forgot-password
/reset-password
```

Elles ne sont pas modifiées.

---

3. Routes accessibles à tous les utilisateurs connectés

Ces routes sont accessibles après login, quel que soit le rôle :

```
GET  /dashboard              # tableau de bord selon le rôle
```

Laravel redirigera selon le rôle (admin, médecin, patient).

---

4. Routes pour le patient

Le patient peut :

- voir les services,
- réserver un service,
- consulter ses réservations,
- annuler une réservation (si elle est en attente).

Routes Patient

```
# Réservation d'un service
GET    /reservation/{service_id}/create     # page du formulaire de réservation
POST   /reservation/store                   # enregistrement de la réservation

# Tableau des réservations
GET    /mes-reservations                    # liste des réservations du patient
```

```
# Annulation
POST   /reservation/{id}/cancel            # annuler une réservation (en attente)
```

Très simple et parfaitement dans le cadre.

---

5. Routes pour le médecin

Le médecin peut :

- voir les services qui lui sont attribués,
- voir ses réservations,
- mettre à jour les statuts.

Routes Médecin

```
GET    /medecin/services               # liste des services appartenant au
médecin
GET    /medecin/reservations           # liste des réservations liées à ses
services

# Mise à jour du statut de réservation
POST   /medecin/reservations/{id}/update-status
```

Statuts possibles :

- confirmée
- annulée
- effectuée

---

6. Routes pour l'administrateur

L'admin peut :

- gérer les services (CRUD complet),
- affecter un médecin,
- gérer les utilisateurs,
- voir toutes les réservations,
- annuler une réservation.

Routes Services (Admin)

```
GET    /admin/services                    # liste
GET    /admin/services/create             # formulaire création
```

```
POST   /admin/services/store              # enregistrer service
GET    /admin/services/{id}/edit          # formulaire modification
POST   /admin/services/{id}/update        # mise à jour
POST   /admin/services/{id}/delete        # suppression
POST   /admin/services/{id}/assign        # assigner un médecin
```

Routes Réservations (Admin)

```
GET    /admin/reservations                # toutes les réservations
POST   /admin/reservations/{id}/cancel    # annuler une réservation
```

Routes Utilisateurs (Admin)

```
GET    /admin/users                       # liste des utilisateurs
POST   /admin/users/{id}/set-role         # changer le rôle (admin / medecin /
patient)
POST   /admin/users/{id}/toggle           # activer / désactiver utilisateur
```

---

7. Fichier complet `routes/web.php` attendu

Voici **le fichier final** que l'application doit respecter.
Il utilise les middlewares `auth`, `role:xxx`, et reste très simple.

```php
<?php

use Illuminate\Support\Facades\Route;

/* ----------------------------------------------------
|  Routes publiques
-----------------------------------------------------*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('/services', [ServiceController::class, 'index']);
Route::get('/services/{id}', [ServiceController::class, 'show']);


/* ----------------------------------------------------
|  Routes Auth (Breeze gère)
-----------------------------------------------------*/

require __DIR__.'/auth.php';
```

```
/* ----------------------------------------------------
|  Tableau de bord général
-----------------------------------------------------*/

Route::get('/dashboard', [DashboardController::class, 'index'])
    ->middleware('auth')
    ->name('dashboard');


/* ----------------------------------------------------
|  Routes patient
-----------------------------------------------------*/

Route::middleware(['auth', 'role:patient'])->group(function () {

    Route::get('/reservation/{service_id}/create', [ReservationController::class,
'create']);
    Route::post('/reservation/store', [ReservationController::class, 'store']);

    Route::get('/mes-reservations', [ReservationController::class,
'myReservations']);
    Route::post('/reservation/{id}/cancel', [ReservationController::class,
'cancel']);
});


/* ----------------------------------------------------
|  Routes médecin
-----------------------------------------------------*/

Route::middleware(['auth', 'role:medecin'])->group(function () {

    Route::get('/medecin/services', [MedecinController::class, 'services']);
    Route::get('/medecin/reservations', [MedecinController::class,
'reservations']);

    Route::post('/medecin/reservations/{id}/update-status',
        [MedecinController::class, 'updateStatus']);
});


/* ----------------------------------------------------
|  Routes administrateur
-----------------------------------------------------*/

Route::middleware(['auth', 'role:admin'])->group(function () {

    // Services
    Route::get('/admin/services', [AdminServiceController::class, 'index']);
    Route::get('/admin/services/create', [AdminServiceController::class,
'create']);
    Route::post('/admin/services/store', [AdminServiceController::class,
```

```
'store']);
    Route::get('/admin/services/{id}/edit', [AdminServiceController::class,
'edit']);
    Route::post('/admin/services/{id}/update', [AdminServiceController::class,
'update']);
    Route::post('/admin/services/{id}/delete', [AdminServiceController::class,
'destroy']);
    Route::post('/admin/services/{id}/assign', [AdminServiceController::class,
'assignMedecin']);

    // Réservations
    Route::get('/admin/reservations', [AdminReservationController::class,
'index']);
    Route::post('/admin/reservations/{id}/cancel',
[AdminReservationController::class, 'cancel']);

    // Utilisateurs
    Route::get('/admin/users', [AdminUserController::class, 'index']);
    Route::post('/admin/users/{id}/set-role', [AdminUserController::class,
'setRole']);
    Route::post('/admin/users/{id}/toggle', [AdminUserController::class,
'toggle']);
});
```

---

8. Points clés à retenir

- Toutes les routes sont **groupées par rôle** pour plus de clarté.
- Aucun étudiant ne doit ajouter d'autres routes.
- Le projet fil rouge doit suivre **exactement** cette structure.
- Les middlewares de rôle doivent être implémentés.
- Le Dashboard redirige selon le rôle.